Proyecto de Curso: Arquitectura de Software 2026
Entrega No. 2: Migración a Microservicios, Integración y Resiliencia

Prof. Nicolás Ramírez Vélez

Semestre 2026-I

Alineación ABET (Acreditación Internacional)

Resultado del Programa: Desarrolla una solución final que satisface todos los requerimientos y restricciones identificadas al formular el problema de diseño.
Resultado de la Asignatura: Implementa la arquitectura de software apropiada (cliente-servidor, por capas, MVC, orientadas a servicios y orientadas a microservicios).

1. Objetivo de la Entrega

Evolucionar la arquitectura monolítica de la primera entrega hacia un ecosistema híbrido y resiliente desplegado en **AWS Academy**. Los equipos aplicarán el **Strangler Pattern** para extraer funcionalidades a microservicios, orquestarán el tráfico mediante un **API Gateway (Nginx)**, e implementarán comunicación asíncrona garantizando estándares de usabilidad e internacionalización.

2. Requerimientos Técnicos (Grado Empresarial)

2.1. 1. Despliegue e Infraestructura (AWS Academy)

- Todo el ecosistema (Django, Flask, Nginx, Base de Datos, Message Broker) debe estar empaquetado y orquestado con **Docker Compose**.
- El sistema debe desplegarse en una instancia EC2 de **AWS Academy**.

2.2. 2. Arquitectura y Patrón Estrangulador

- Aísle funcionalidades del monolito (Django) y mígrelas a microservicios independientes (Flask).
- Configure **Nginx** como API Gateway para el ruteo de tráfico profesional.

2.3. 3. Integración y Patrones Estructurales

- **Servicio a Proveer:** Exponga un endpoint JSON con información relevante del sistema.
- **Servicio a Consumir:** Consuma el servicio del *equipo aliado* y muéstrelo en su aplicación.
- **API de Terceros:** Consumo obligatorio mediante el **Patrón Adapter** (Inversión de Dependencias).

2.4. 4. Comunicación Asíncrona e i18n

- **Eventos:** Uso de *Message Broker* (Redis/Celery) para procesos de fondo (reportes, notificaciones).
- **Usabilidad:** Auditoría de UI/UX, navegación y formularios robustos.
- **i18n:** Soporte bilingüe completo mediante `gettext`. Prohibido el uso de textos quemados.

3. Rúbrica de Evaluación Institucional (ABET)

ADVERTENCIA CRÍTICA

De acuerdo con los lineamientos del curso: **Si el equipo no realiza el 100 % de las correcciones notificadas en la Entrega 1, la nueva entrega NO será revisada y la nota será 0.0**.

Criterio	Descripción	Peso
Correcciones E1	Implementación total de ajustes indicados en la sustentación previa.	10 %
Arquitectura	Actualización de diagramas incluyendo AWS, Nginx, Flask y Broker.	10 %
Servicios	Exposición de API, consumo de aliado y consumo de tercero vía Adapter.	30 %
Infraestructura	Despliegue en AWS Academy con Docker y ruteo vía Nginx Gateway.	30 %
Resiliencia y UX	Implementación de i18n, usabilidad y tareas asíncronas asíncronas.	20 %

Cuadro 1: Matriz de Evaluación - Entregable 2

4. Bono de Excelencia Arquitectónica

Exención de Parcial 02

Aquellos equipos que logren llevar su arquitectura más allá del requerimiento mínimo y migren **más del 70 % de sus servicios y lógica de negocio a microservicios funcionales e independientes**, quedarán automáticamente **eximidos de presentar el Parcial 02** de la asignatura.

5. Instrucciones de Entrega

1. Comparta el enlace del repositorio (rama `main`) y la IP elástica de AWS en la plataforma institucional.
2. **Evidencia de Trabajo:** Estudiante sin *commits* en el repositorio tiene nota de 0.0.
3. La instancia de **AWS EC2** debe estar encendida para la sustentación.